DataShare

Documentation technique du prototype (MVP) + US09

Plateforme de transfert sécurisé de fichiers pour freelances et petites entreprises

Information	Valeur
Auteur	GNANGUY Coffi Désiré
Destinataire	Lisa, responsable produit
Version / date	1.0 — 06 novembre 2026
Dépôt Git	https://github.com/Mrflex-225/datashare-backend.git https://github.com/Mrflex-225/datashare-frontend.git

Table des matières

Architecture de l'application (diagramme simple) 3
Légende et flux principaux 3
Choix technologiques justifiés 4
Modèle de données 5
Documentation d'API 6
Endpoints principaux 6
Sécurité et gestion des accès 6
Authentification 6
Autorisations 7
Protection des mots de passe 7
Limites et protections 7
Qualité, tests et maintenance 8
Processus d'installation et d'exécution 8
Prérequis 8
Installation et lancement 8
Variables d'environnement 9
Vérification 9
Utilisation de l'IA dans le développement 9
Posture adoptée 9
Tâches confiées à l'IA 9
Supervision et vérification 9
Apports et limites 10

Architecture de l'application (diagramme simple)

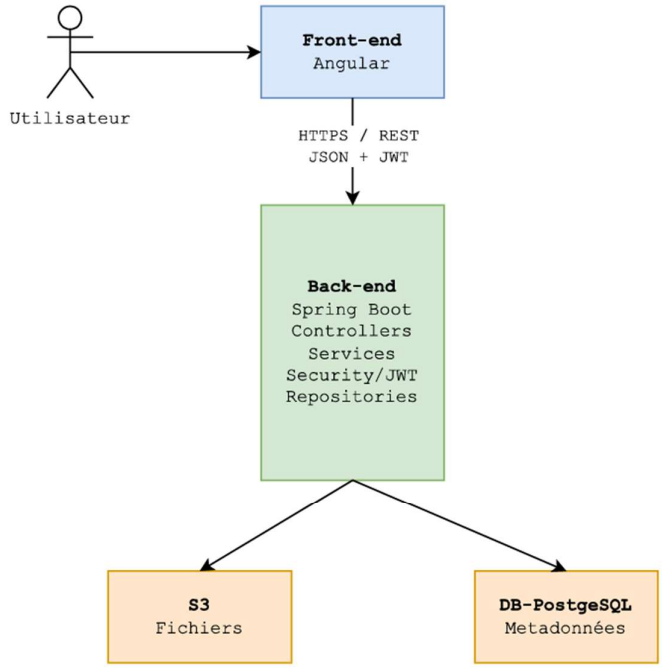


Figure 1 — Architecture globale

Légende et flux principaux

- **Frontend -> Backend**: échange applicatif. Tous les échanges entre le navigateur et le serveur passent en HTTPS ; les appels protégés portent un JWT dans l'en-tête Authorization: Bearer.
- **Bakcend -> AWS** : accès direct à S3 par URL pré-signée à durée limitée. Le bucket reste privé.
- **Téléversement** : Angular envoie le fichier en multipart au backend, qui valide la taille et le type, le stocke dans S3, puis enregistre les métadonnées dans PostgreSQL.
- **Partage** : le backend génère un token unique ; le lien /download/{token} peut être transmis à une personne sans compte.
- **Téléchargement** : le backend vérifie l'existence du fichier, la date d'expiration et, si besoin, le mot de passe du lien, puis délivre une URL pré-signée S3 temporaire.

Choix technologiques justifiés

Les choix privilégient la robustesse, la maintenabilité et la rapidité de livraison dans le délai de 4 semaines, en s'appuyant sur des technologies maîtrisées et largement documentées.

Élément	Technologie choisie	Alternatives	Justification
Langage backend	Java 21	Python, C#, JavaScript	Robustesse et typage fort ; maîtrise acquise sur des projets précédents, ce qui réduit la courbe d'apprentissage et sécurise le délai.
Framework backend	Spring Boot	Django, NestJS, ASP.NET Core	Architecture structurée et modulaire, adaptée aux environnements d'entreprise. Intégration native de Spring Security, Spring Data JPA et des outils de test.
Langage frontend	TypeScript	JavaScript	Typage statique : erreurs détectées à la compilation, code plus lisible et plus fiable à maintenir.
Framework frontend	Angular	React, Vue.js	Architecture structurée et écosystème intégré (routeur, formulaires, HTTP). Approche composants/services favorable à la modularité et à la réutilisation.
Base de données	PostgreSQL	MySQL, MongoDB	Fiabilité, propriétés ACID, et gestion adaptée des relations entre utilisateurs, fichiers et tags.
Stockage des fichiers	AWS S3	Stockage local, Azure Blob, MinIO	Stockage indépendant du serveur applicatif, durable et évolutif. Les URL pré-signées offrent des téléchargements temporaires sans bucket public.
Authentification	Spring Security + JWT	Sessions HTTP, OAuth2/OIDC	Contrôle d'accès centralisé côté API. JWT : authentification stateless simple entre Angular et Spring Boot.
Accès aux données	Spring Data JPA / Hibernate	JDBC	Repositories et mapping objet-relationnel : moins de code répétitif, requêtes paramétrées (protection contre l'injection SQL).
Conteneurisation	Docker / Docker Compose	Installation locale des services	Environnement reproductible (notamment PostgreSQL) ; moins de différences entre machines, démarrage simplifié.
Gestion des dépendances	Maven	Gradle	Dépendances centralisées, cycle de build standardisé, builds reproductibles.
Versionnement	Git (Conventional Commits)	SVN, Mercurial	Branches, traçabilité et retour arrière. Messages de commit normalisés pour un historique lisible.
Tests backend	JUnit 5, Mockito, MockMvc	TestNG, tests manuels	Tests de la logique métier, isolation des dépendances et vérification des endpoints REST ; détection des régressions.
Tests frontend	Jest (unitaires) + Cypress (E2E)	Karma/Jasmine, Playwright	Jest : exécution rapide sans navigateur réel, mocks intégrés, pour tester composants, services, intercepteur JWT et guards de routes. Cypress : tests de bout en bout dans un vrai navigateur, pour valider le parcours complet (connexion, upload, lien, téléchargement) tel que vu par l'utilisateur.

Élément	Technologie choisie	Alternatives	Justification
Assistant de code	Claude Code	Autres copilotes (GitHub Copilot, Cursor), développement sans assistance	Agent en ligne de commande qui travaille directement dans le dépôt : il lit le code, modifie les fichiers et exécute les tests. Il accélère l'implémentation et la génération de tests ; chaque modification est relue (git diff) avant d'être validée (voir section 8).

Modèle de données

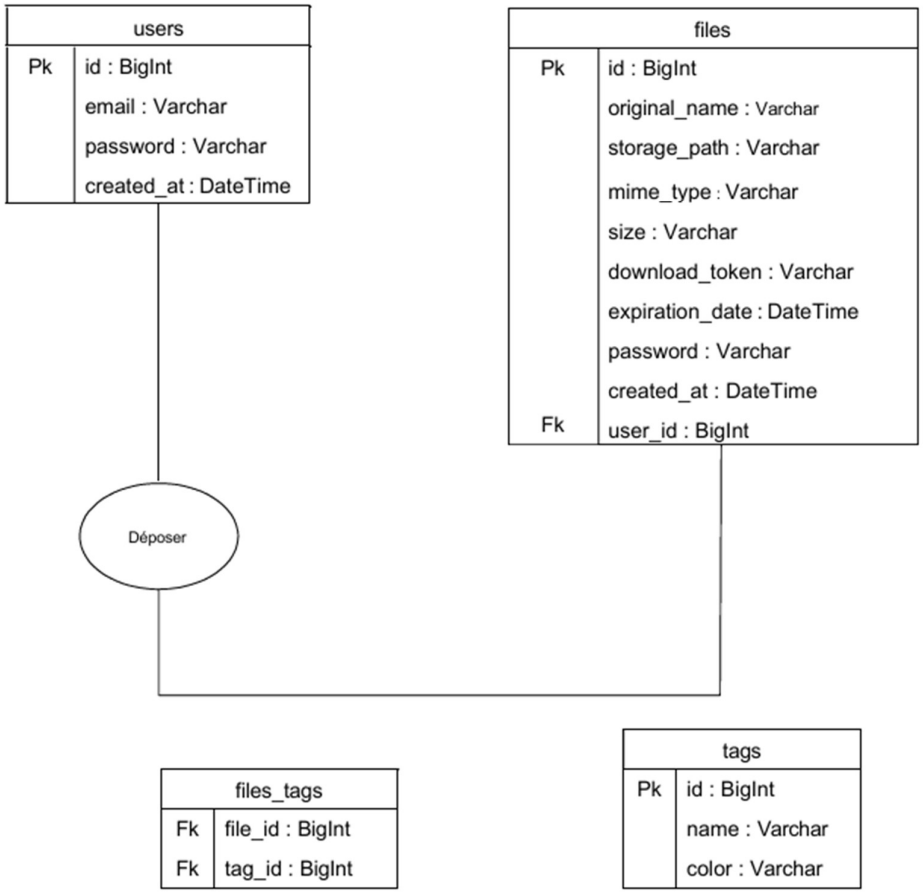


Figure 2 — Diagramme entité-relation (PK : clé primaire, FK : clé étrangère)

Documentation d'API

Le contrat d'interface entre Angular et Spring Boot est décrit au format **OpenAPI 3**. La spécification est générée par springdoc-openapi et consultable en exécution à /v3/api-docs (JSON) et /swagger-ui.html (interface interactive) apres avoir lancé le backend.

Endpoints principaux

Méthode	Route	Accès	Description et réponses
POST	/api/register	Public	Corps JSON validé (@Valid) : email, mot de passe. 201 sans corps ; 400 si données invalides ; 409 si email déjà utilisé.
POST	/api/login	Public	Corps JSON : email, password. 200 avec le JWT dans le corps de la réponse (texte brut) ; 401 si identifiants invalides.
POST	/api/files	JWT	multipart/form-data : file (obligatoire), password (optionnel), expirationDays (optionnel). 201 + métadonnées du fichier (FileMetadataDTO) incluant le lien de partage ; erreur si fichier refusé.
GET	/api/files/me	JWT	Historique des fichiers de l'utilisateur connecté, identifié par son JWT (FileHistoryDTO). 200.
DELETE	/api/files/{id}	JWT	Supprime un fichier du propriétaire (base + S3). 204 ; refus si non propriétaire ou fichier inexistant.
GET	/api/files/download/{token}/metadata	Public	Informations publiques du lien (PublicFileMetadataDTO), utilisées par la page de téléchargement. 200 ; 404 si inconnu ; 410 si expiré.
POST	/api/files/download/{token}	Public	Corps JSON optionnel { "password": "..." }. Vérifie le lien et le mot de passe, puis renvoie { "signedUrl": "..." } (URL S3 pré-signée, durée limitée). 200 ; 401/403 si mot de passe incorrect ; 404/410 si inconnu ou expiré.

Sécurité et gestion des accès

La sécurité est un aspect essentiel de DataShare, puisque l'application permet de stocker et de partager des fichiers potentiellement confidentiels. Plusieurs mécanismes protègent les données, contrôlent les accès et limitent les risques liés au téléversement et au téléchargement.

Authentification

L'authentification repose sur Spring Security et les JSON Web Tokens (JWT). Après une connexion valide, le backend génère un token que le frontend Angular transmet dans l'en-tête HTTP Authorization (Bearer Token). Un intercepteur HTTP Angular l'ajoute automatiquement aux requêtes protégées.

Côté backend, un filtre Spring Security vérifie la validité du JWT (signature, expiration) avant d'autoriser l'accès aux ressources protégées. L'approche est stateless : aucune session serveur n'est conservée.

Autorisations

DataShare distingue les utilisateurs authentifiés des visiteurs non authentifiés. Chaque fichier est associé à son propriétaire (relation File–User), ce qui permet de contrôler la consultation de l'historique et la suppression.

Le téléchargement est une exception volontaire : un utilisateur peut partager un fichier avec une personne sans compte grâce à un lien contenant un token unique et difficile à deviner. Avant d'autoriser le téléchargement, le backend vérifie l'existence du fichier, la validité du lien, sa date d'expiration et, si un mot de passe a été défini, une vérification supplémentaire.

Protection des mots de passe

Les mots de passe ne sont jamais conservés en clair dans PostgreSQL. Ils sont hachés avec BCrypt, intégré à Spring Security. Le hachage est à sens unique : lors de la connexion, l'application compare la valeur saisie au hash enregistré sans jamais retrouver le mot de passe initial. Il en va de même pour les mots de passe de liens de partage.

Limites et protections

Risque	Protection
Fichiers trop volumineux	Taille maximale de fichier et de requête configurée côté Spring (spring.servlet.multipart) ; valeur : 1GB.
Types de fichiers indésirables	Contrôle du type/de l'extension à l'upload ; ("exe", "bat", "cmd", "com"…)
Devinette de liens	Token aléatoire long et unique ; réponse 404 sans fuite d'information.
Accès prolongé aux fichiers	Expiration des liens contrôlée côté serveur ; URL S3 pré-signées de courte durée.
Fuite de fichiers	Bucket S3 privé ; aucun accès public.
Injection	Requêtes JPA paramétrées ; validation des entrées avec Bean Validation ; Angular échappe le contenu affiché.
Secrets	Clés AWS, secret JWT et identifiants BDD fournis par variables d'environnement ; jamais commités.

Qualité, tests et maintenance

Fichier	Contenu	Résultats clés vérifiés
TESTING.md	Tests unitaires et d'intégration du backend (JUnit 5, Mockito, MockMvc), tests frontend Jest et parcours E2E Cypress. Mesure de couverture JaCoCo.	Backend : 66/66 tests réussis (0 échec, 0 erreur, 0 ignoré). Frontend : 90 tests Jest réussis. E2E : 23 tests Cypress réussis. JaCoCo : 91 % des instructions et 80 % des branches couvertes.
SECURITY.md	Contrôles d'accès, analyse des dépendances npm audit, essais de scans Trivy et OWASP Dependency-Check, suivi des alertes.	npm audit : 24 alertes (8 faibles, 16 modérées, 0 élevées, 0 critiques). npm audit --omit=dev : 0 vulnérabilité. Trivy : résultat non exploitable (aucune cible compatible détectée). OWASP : analyse non finalisée (accès NVD).
PERF.md	Test de performance de l'endpoint de génération de lien de téléchargement avec Grafana k6. Mesure du temps de réponse p95, du taux d'erreur HTTP et du nombre d'utilisateurs virtuels. Analyse des résultats et présentation des limites du test. Évaluation des performances frontend avec Lighthouse prévue.	Un test k6 a été exécuté et validé avec 1 utilisateur virtuel et 1 requête HTTP. Temps de réponse : 46,82 ms, taux d'erreur : 0 %, vérifications réussies : 100 %. Le test de charge avec plusieurs utilisateurs reste à valider. Aucune mesure Lighthouse vérifiée à ce stade.
MAINTENANCE.md	Documentation des procédures de maintenance : mise à jour des dépendances frontend et backend, gestion des fichiers expirés, sauvegarde et restauration de PostgreSQL, rotation des secrets et vérifications après intervention.	Les procédures de maintenance sont documentées. Leur exécution opérationnelle reste à vérifier, notamment pour les sauvegardes, la restauration et la rotation des secrets. Un suivi mensuel des dépendances et une revue périodique des sauvegardes et des accès sont proposés.

Processus d'installation et d'exécution

Prérequis

Outil	Version	Usage
Java JDK	21	Backend Spring Boot (4.1.1)
Maven	Wrapper mvnw fourni	Build backend, sans installation globale
Node.js / npm	24.9.0 / 11.6.0	Frontend Angular 21
Docker Desktop	compatible Docker Compose	PostgreSQL 16 en conteneur
Compte AWS	bucket S3 privé	Stockage des fichiers

Installation et lancement

```
1. Base de données PostgreSQL (dossier contenant docker-compose.yml)
docker compose up -d

2. Backend, depuis datashare-backend  ->  http://localhost:8080
./mvnw clean install
./mvnw spring-boot:run          # Windows : .\mvnw.cmd spring-boot:run

3. Frontend, depuis le dossier Angular  ->  http://localhost:4200
npm install
npm start
```

Le backend et le frontend doivent tourner en même temps.

Variables d'environnement

Variable	Rôle
DB_URL/ DB_USER / DB_HOST / DB_PASSWORD/ DB_NAME/ DB_PORT	Connexion à PostgreSQL
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Identifiants d'accès à S3
AWS_REGION / AWS_S3_BUCKET	Région et nom du bucket
JWT_SECRET	Clé de signature des tokens JWT

Ces valeurs ne sont jamais enregistrées dans le code ni dans Git.

Vérification

Créer un compte et se connecter, téléverser un fichier, utiliser le lien de téléchargement généré, consulter l'historique, puis supprimer le fichier : ce parcours valide la chaîne Angular, Spring Boot, PostgreSQL et S3.

Utilisation de l'IA dans le développement

Deux outils ont été utilisés : **Claude Code** (Opus 5.5) et **ChatGPT** (GPT-6).

Posture adoptée

Approche **hybride** : délégation de tâches précises à Claude Code, utilisé comme un développeur junior à qui l'on donne des consignes et un résultat attendu ; échanges en binôme avec ChatGPT pour comprendre des notions, comparer des solutions et découvrir des outils.

Tâches confiées à l'IA

Outil	Tâches
Claude Code	US01 (téléversement) : interface Angular réalisée d'après la maquette Figma, et connexion au backend.
ChatGPT	Conception de l'endpoint d'upload ; aide à la rédaction des tests front et back ; prise en main de l'audit des dépendances front et de k6 ; conseils d'amélioration et d'organisation du travail.

Supervision et vérification

- Relecture et compréhension de tout code généré avant intégration ; explications demandées quand un point restait flou.

- Tests manuels et exécution des tests automatisés pour contrôler le comportement réel.
- Vérification que les tests générés correspondent au comportement attendu, et pas seulement qu'ils passent.

Apports et limites

- **Apports :** gain de temps, notamment sur l'interface Angular issue de la maquette et sur les tâches répétitives ; montée en compétence sur les tests, l'audit de sécurité et k6.
- **Limites :** réponses à vérifier (une proposition plausible peut être inadaptée) ; contexte du projet incomplet, d'où des consignes précises à fournir ; solutions parfois plus complexes que nécessaire.

Bilan : l'IA accélère le travail, mais c'est au développeur de comprendre, tester et valider ce qu'il intègre.